MAGNUS SIG

Proyecto final, documentación y sustentación

Versión académica 1.0

Asignatura: Lenguaje de Programación 1

Periodo: 2026-2

Integrantes: Juan José Osorio Quitian · Kevin Julián Sánchez Pérez · Brayan Steven Sánchez Carvajal · Luis Esteban Medina Echeverry · Hilver Eduardo Galindo

Docente: dato pendiente de confirmar

Neiva, Huila · 2026

Tabla de contenido

1. Presentación del proyecto

2. Problema, objetivos y alcance

3. Requisitos y funciones principales

4. Arquitectura y modelo de datos

5. Metodología de desarrollo

6. Calidad y resultados de pruebas

7. Guía breve de usuario

8. Retos y conclusiones

9. Sustentación y enlaces

Referencias

1. Presentación del proyecto

Magnus SIG es una aplicación web para administrar inventario de equipos de protección personal, dotación y suministros relacionados con seguridad y salud en el trabajo. El sistema permite gestionar catálogo y categorías, existencias, movimientos, compras y despachos, proveedores, clientes, servicios, reportes y configuración. La versión revisada corresponde a una aplicación Django organizada con el patrón Modelo-Vista-Plantilla (MVT).

Tecnologías y organización

- Backend: Python 3.14 en el entorno verificado y Django 6.1.1.
- Persistencia: MySQL configurado por defecto; SQLite se puede activar con DJANGO_USE_SQLITE.
- Interfaz: plantillas Django, Tailwind CSS 4, JavaScript y HTMX; las dependencias de movimiento y visualización se empaquetan con npm/esbuild.
- Control de acceso: inicio de sesión con perfiles Administrador y Vendedor y restricciones por módulo.

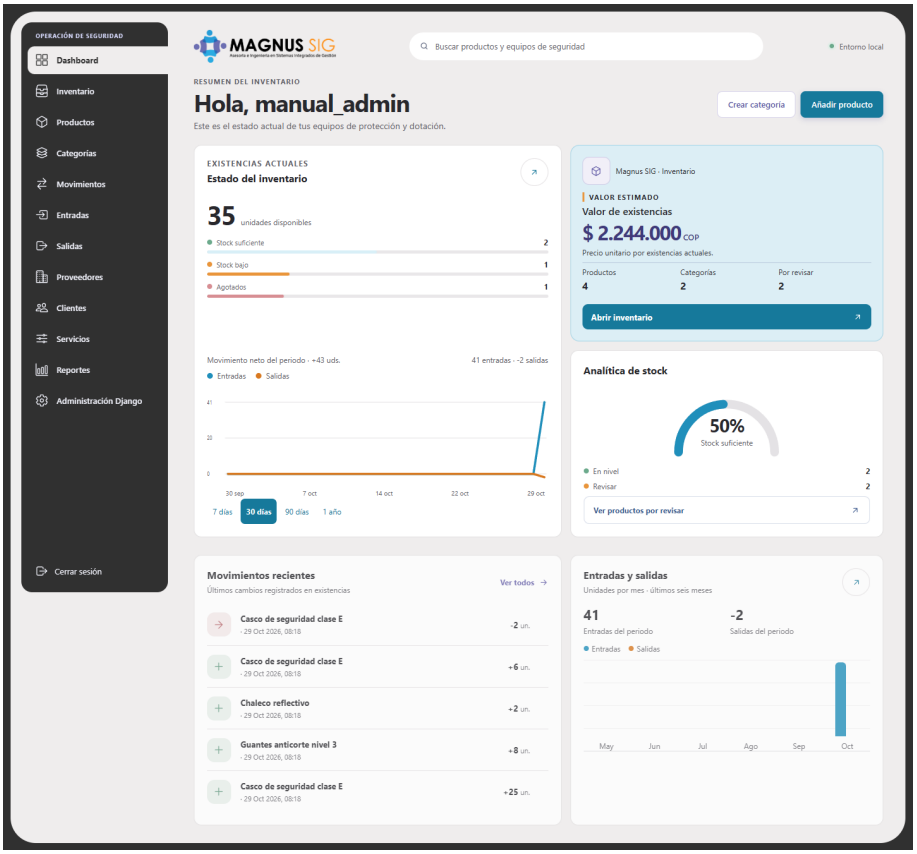


Figura. Panel de administración con indicadores y navegación de módulos.

2. Problema, objetivos y alcance

El proyecto aborda la dispersión de datos y la dificultad para mantener existencias actualizadas en operaciones que registran productos, entradas y despachos. Su objetivo es centralizar esas operaciones y mantener registros consultables de los cambios de inventario.

Funciones incluidas

- Crear, consultar, editar y administrar productos y categorías.

- Consultar existencias y alertas según el stock mínimo configurado.
- Registrar compras/entradas y ventas/despachos mediante estados pendientes y confirmación.
- Conservar movimientos con producto, cantidad, tipo, fecha y responsable.
- Administrar proveedores, clientes y servicios SG-SST.
- Consultar reportes, exportar datos y modificar parámetros globales.

La documentación cubre las funciones del código en la rama main. No atribuye al sistema certificaciones de cumplimiento, inspecciones de EPP ni gestión de incidentes, que no están implementadas.

3. Requisitos y funciones principales

El software se ejecuta localmente con Python/Django y una base de datos MySQL accesible, o con SQLite para un entorno aislado. Para compilar los recursos de frontend se requieren Node.js y npm. El README describe instalación, variables de entorno, migraciones, preparación de roles y creación de superusuario.

Criterios funcionales demostrables

- El usuario inicia sesión antes de acceder a las vistas protegidas.
- El Administrador mantiene el catálogo, proveedores, compras y configuración; el Vendedor consulta inventario y registra despachos dentro de sus permisos.
- Una entrada afecta existencias cuando se confirma su recepción.
- Una salida descuenta existencias al despacharse y se rechaza si no hay unidades suficientes.
- Los formularios validan datos antes de guardar.

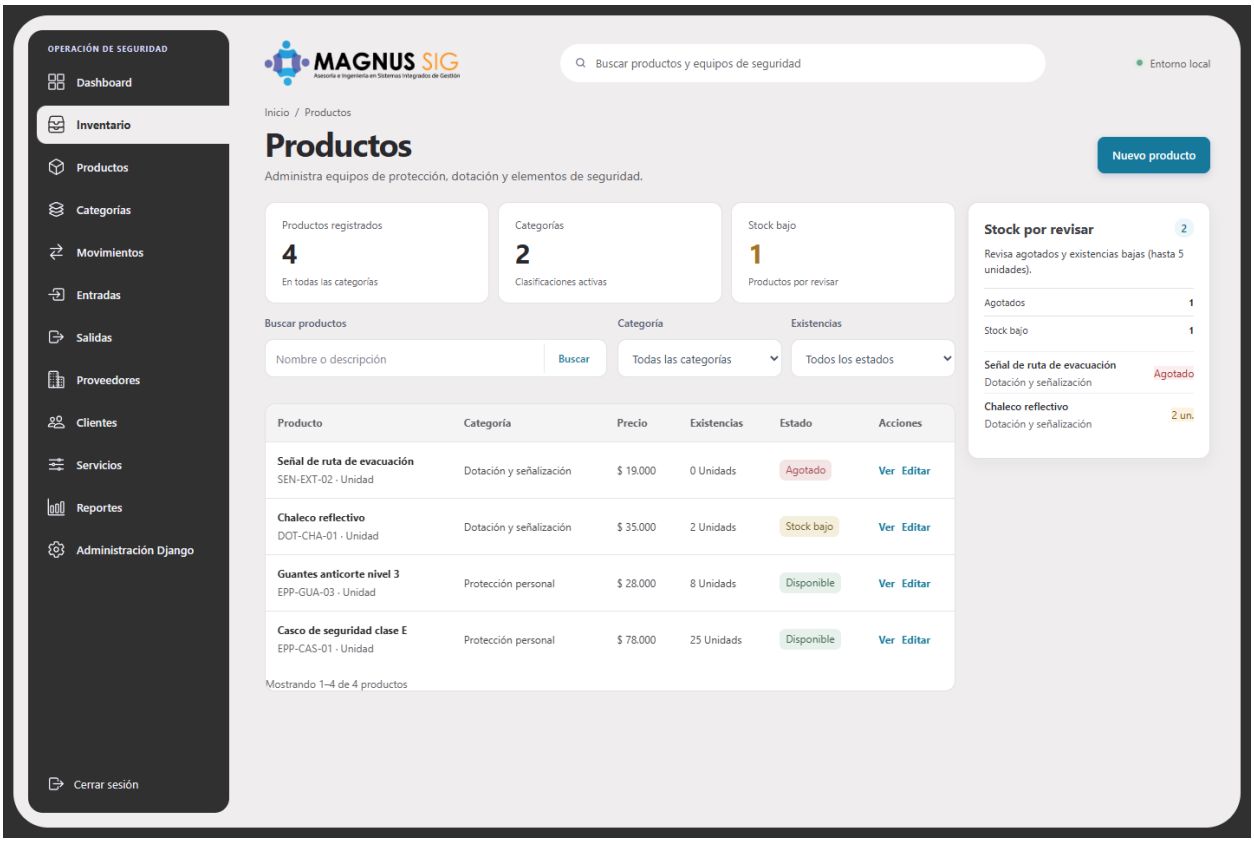


Figura. Consulta de inventario con estado de existencias.

4. Arquitectura y modelo de datos

La capa de presentación usa plantillas HTML y archivos estáticos. Las URL dirigen solicitudes a las vistas; estas aplican permisos y coordinan formularios, modelos y servicios. El servicio de inventario usa transacciones atómicas para registrar la operación y ajustar el saldo. Los modelos Django representan entidades relacionales persistidas mediante el ORM.

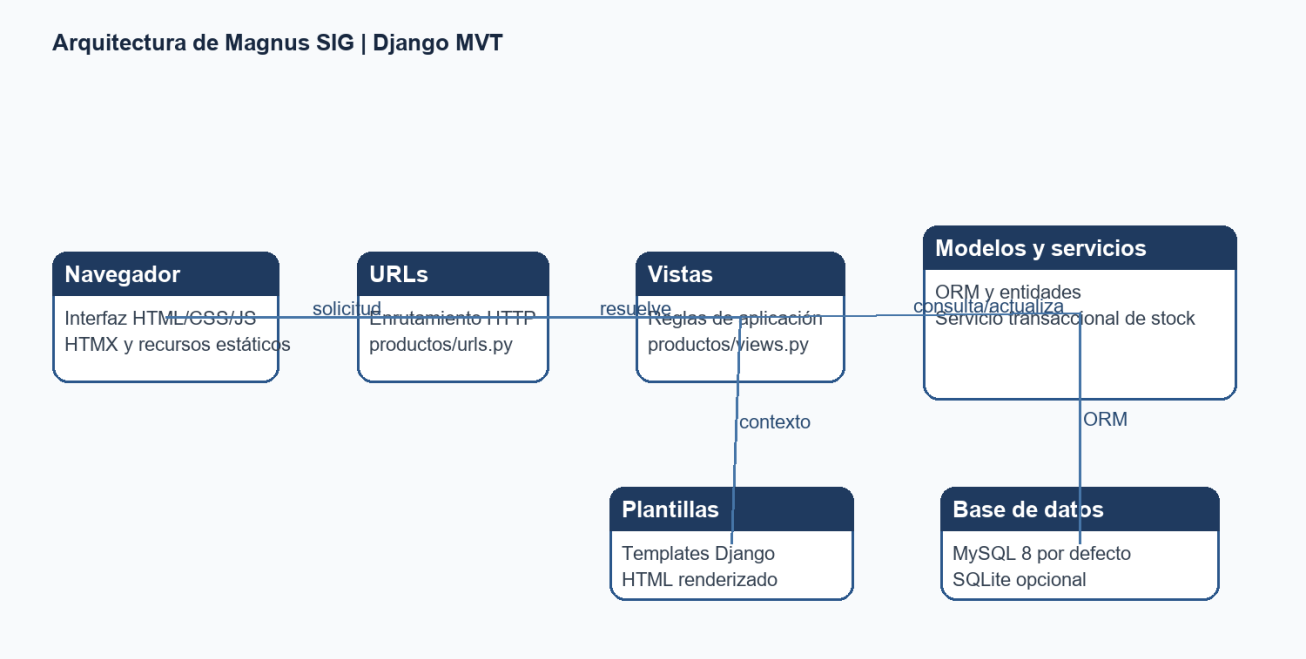


Figura. Diagrama de arquitectura derivado de rutas, vistas, servicios, modelos y configuración del proyecto.

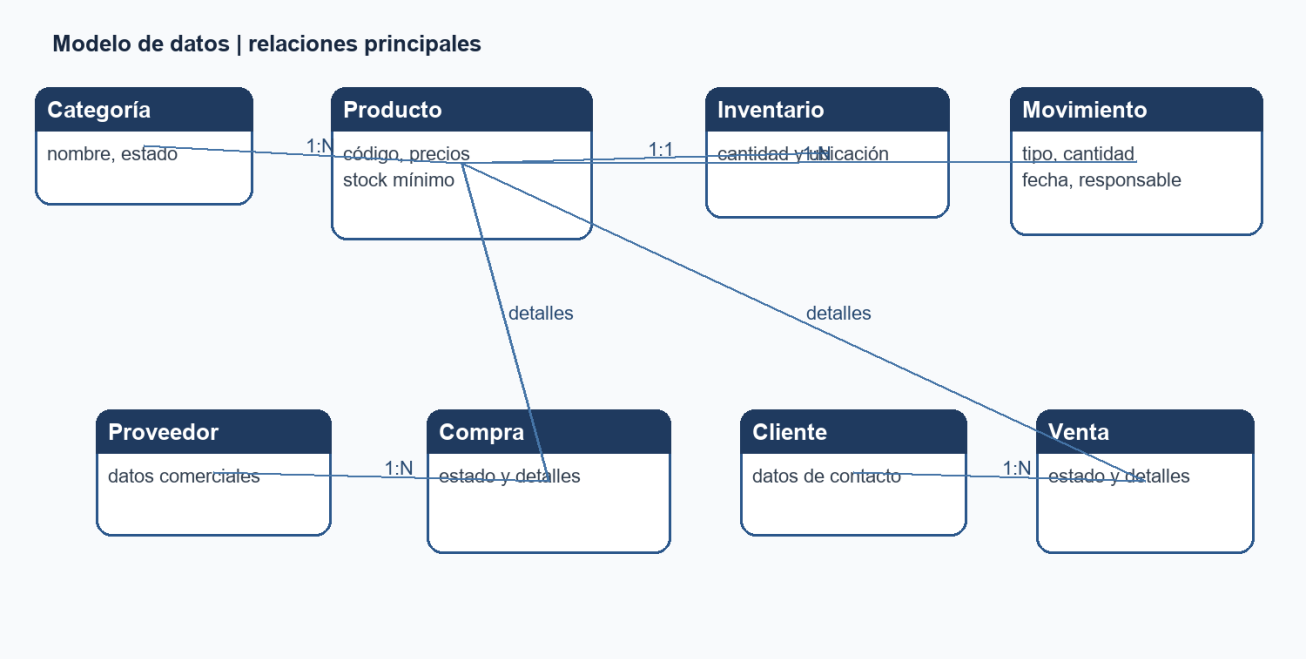


Figura. Relaciones principales observadas en los modelos Django; el diagrama resume las entidades centrales, no todos los campos.

Persistencia y migraciones

La configuración leída para esta revisión conecta por defecto a MySQL, y ambas migraciones de la aplicación productos aparecen aplicadas en esa conexión. El aviso de una migración pendiente visto en otra terminal puede corresponder a otra base o configuración local. Para comprobarlo, ejecutar showmigrations con las mismas variables de entorno y el mismo archivo .env que usa runserver.

5. Metodología de desarrollo

El equipo organizó el trabajo en seis Sprints y utilizó Kanban para visualizar el estado de las tareas. La secuencia documentada comienza con el entorno y el MVP de consola, continúa con el modelo de datos, operaciones CRUD e interfaz, y termina con pruebas y documentación.

Sprint	Propósito
1	Preparación del entorno y prototipo inicial
2	Modelo y persistencia
3	CRUD y validaciones
4	Interfaz y experiencia de usuario
5	Pruebas funcionales
6	Cierre y documentación

La asignación nominal del Product Owner no se incluye porque el dato anterior fue confirmado como incorrecto y el equipo aún debe definir el nombre correcto.

6. Calidad y resultados de pruebas

La revisión ejecutó el chequeo de configuración y la suite de Django sobre la conexión MySQL activa. El chequeo no reportó problemas y las 20 pruebas finalizaron correctamente. Durante la primera ejecución, un caso falló al intentar duplicar un tipo de servicio que la migración ya había creado; el test se ajustó para reutilizar el registro y luego la suite pasó completa.

Evidencia de verificación

Comandos ejecutados: manage.py check; manage.py showmigrations productos; manage.py test productos.

Resultado: 20 pruebas ejecutadas, resultado OK. El tiempo informado por la ejecución final fue aproximadamente 12,9 segundos.

La ejecución de tests crea una base temporal para pruebas y la destruye al terminar. No modifica la base de datos del usuario. La migración 0002 está aplicada en la conexión consultada.

7. Guía breve de usuario

Acceso. Abrir la dirección local de la aplicación e iniciar sesión con las credenciales configuradas por el administrador. Los usuarios y contraseñas no se incluyen en este documento.

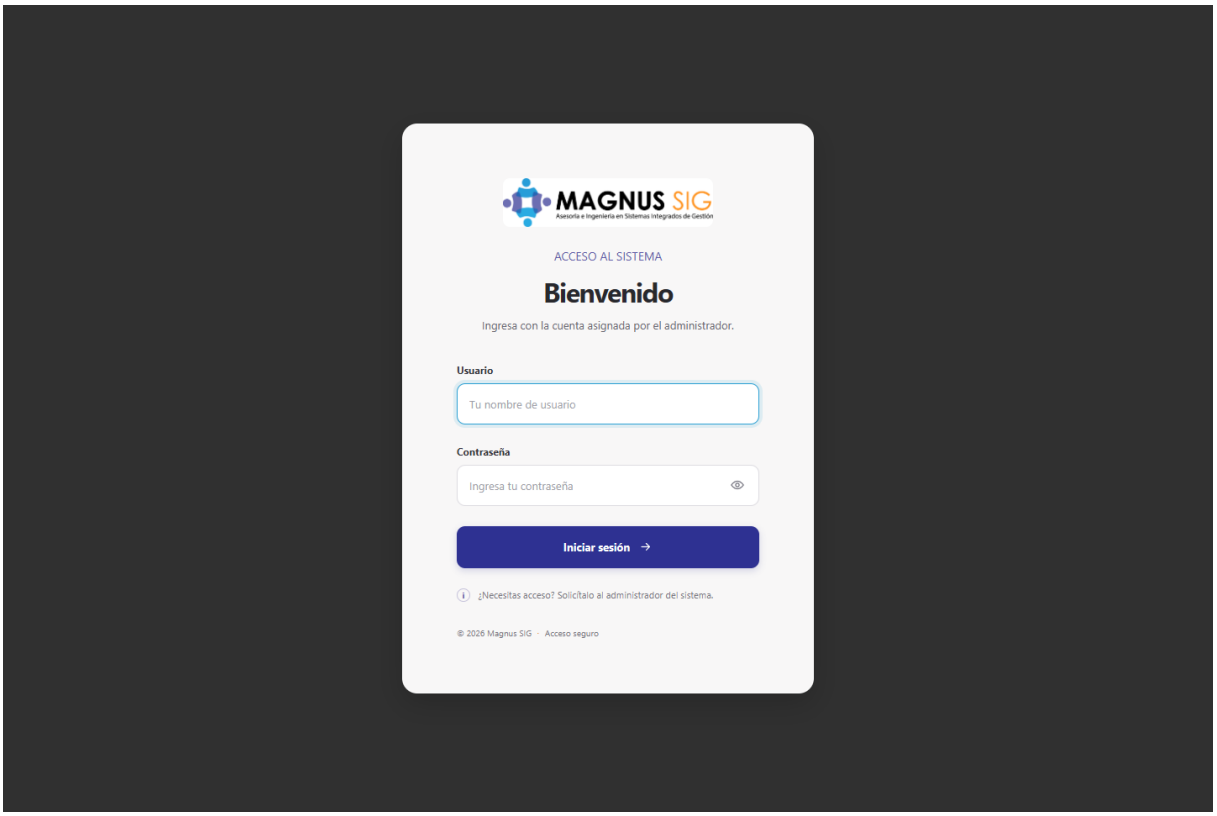


Figura. Acceso al sistema.

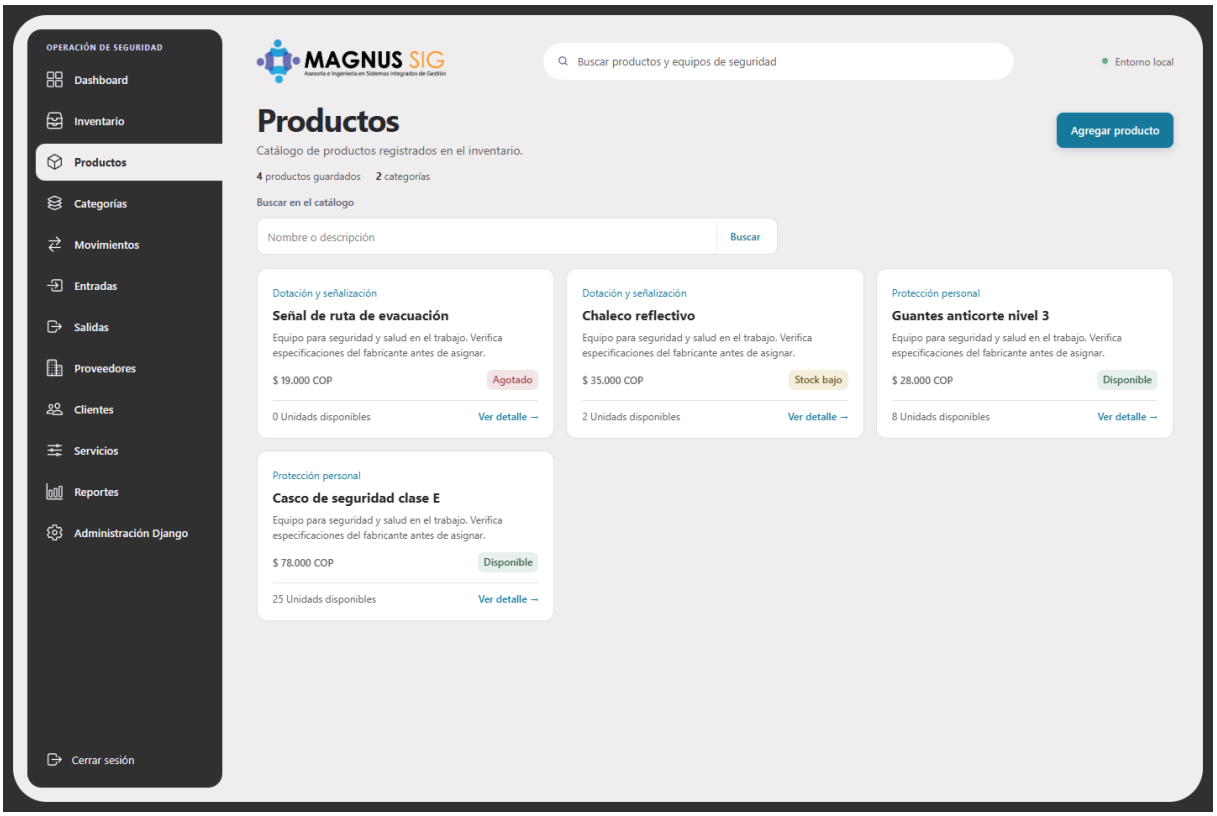


Figura. Catálogo y consulta de productos.

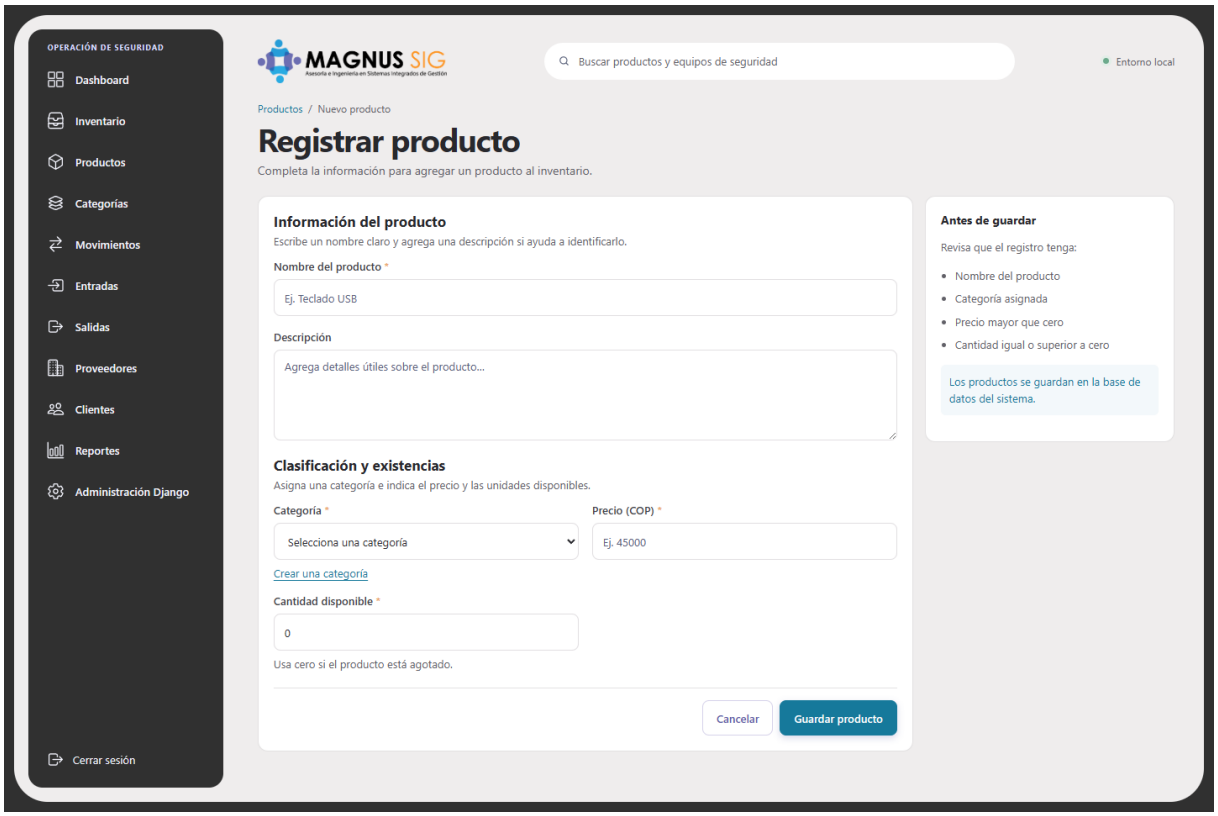


Figura. Formulario para crear o editar productos.

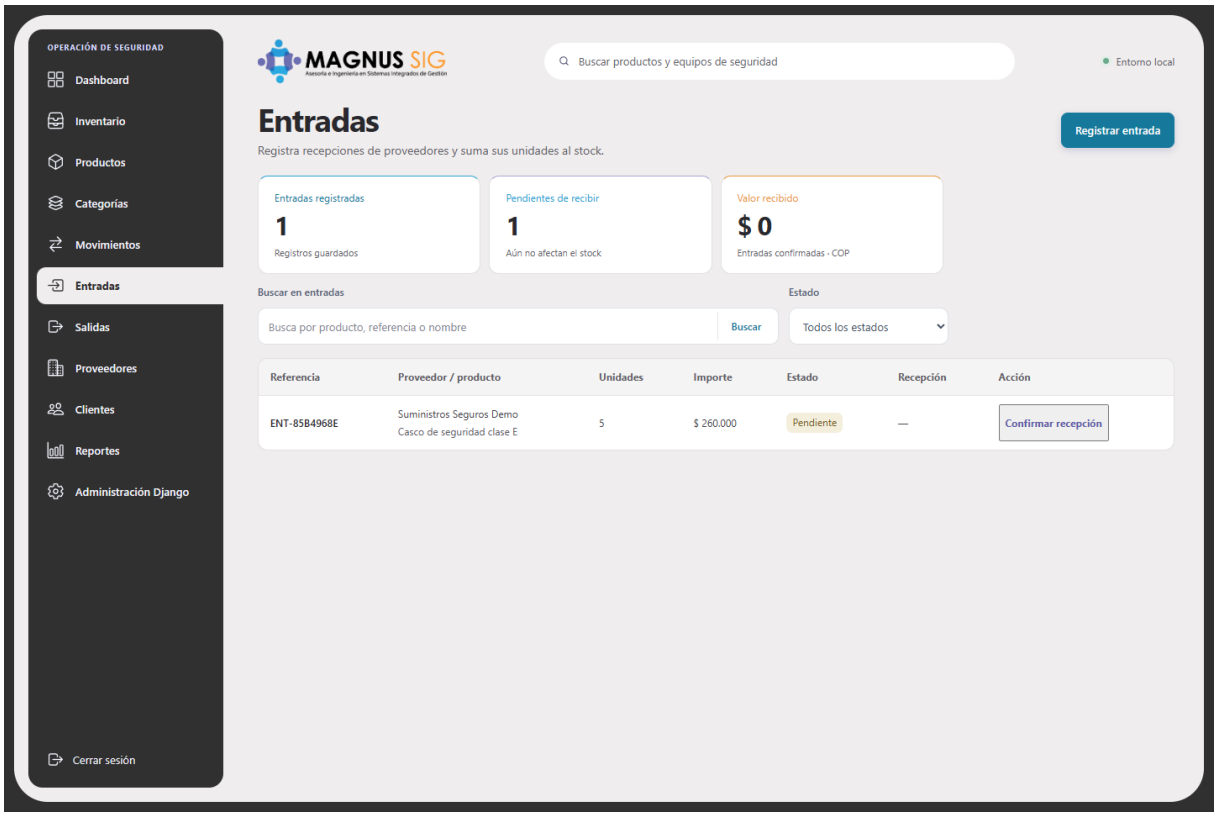


Figura. Listado de entradas de mercancía.

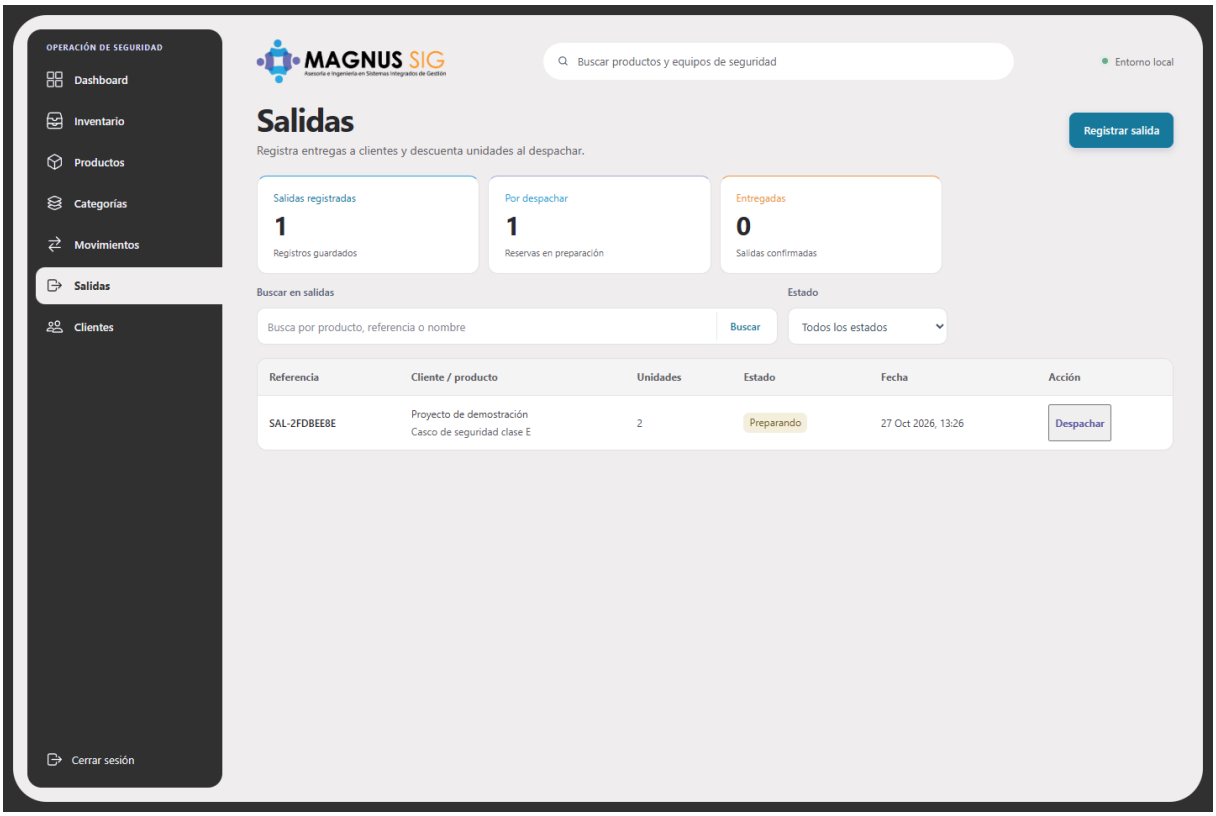


Figura. Módulo de salidas para vendedor.

Flujo de operación de inventario

Para recibir mercancía, el usuario autorizado registra la entrada y confirma la recepción una vez verificada. Para despachar, registra la salida, revisa el cliente, los productos y las cantidades, y confirma el despacho. Si las existencias no alcanzan, el sistema rechaza la operación y presenta un mensaje. El manual completo está disponible en docs/Manual_Usuario_Magnus_SIG.pdf y contiene procedimientos detallados de todos los módulos.

8. Retos y conclusiones

Un reto técnico fue mantener sincronizados los cambios de existencias con el historial de movimientos y evitar saldos negativos. La lógica se concentra en un servicio transaccional que valida cantidades y registra el cambio junto con el movimiento. También se resolvió la convivencia de tipos de servicio base con las pruebas automatizadas, haciendo que los datos iniciales provengan de migraciones y que las pruebas reutilicen esos registros.

El proyecto integra conceptos de programación orientada a objetos, funciones modulares, validaciones, colecciones, persistencia relacional y control de acceso. La suite actual cubre rutas, formularios, entradas, salidas, servicios y permisos. Como cierre, se recomienda mantener la configuración local fuera de Git, revisar las migraciones de cada entorno y documentar futuras mejoras en nuevos incrementos.

9. Sustentación y enlaces

Repositorio público: https://github.com/soykelian/CRUD_MagnusSIG

Video de demostración: pendiente de grabación y publicación. Agregar aquí el enlace compartible antes de cargar el PDF.

Recorrido recomendado para el video

- Iniciar sesión con un usuario de demostración seguro, sin mostrar contraseñas.
- Mostrar dashboard, inventario y búsqueda de productos.
- Registrar o consultar un producto y enseñar una validación de formulario.
- Registrar una entrada y confirmar recepción para evidenciar el aumento de existencias.
- Intentar una salida superior al stock para mostrar el rechazo; luego despachar una salida válida.
- Mostrar el historial de movimientos, permisos de Administrador/Vendedor y un reporte exportado.
- Cerrar con arquitectura, manejo transaccional de stock, pruebas, reto técnico y conclusiones del equipo.

Historial de commits: consultar el registro público de GitHub para evidenciar participación individual. Cada integrante debe poder explicar el módulo en el que colaboró.

Datos de portada por completar antes de entregar: nombre del docente y enlace del video.

Referencias

Django Software Foundation. (s. f.). Django documentation. https://docs.djangoproject.com/

Schwaber, K., & Sutherland, J. (2020). La guía de Scrum. https://scrumguides.org/

Tailwind Labs. (s. f.). Tailwind CSS documentation. https://tailwindcss.com/docs

Magnus SIG. (2026). Manual de usuario Magnus SIG (versión 1.0) [Manual de usuario]. docs/Manual_Usuario_Magnus_SIG.pdf